SITECORE → CONTENTSTACK

# WinnDixie Migration Architecture

How content leaves Sitecore, what the pipeline does to it, what the Contentstack model looks like on the other side, and the rules that decide what is carried. Every diagram is drawn from the actual export and configuration; every count is from the current build and verified on the stack.

| | | | |
|---|---|---|---|
| **4,686** entries in the export | **2,082** assets (media files) | **56** content types with entries | **10** global fields |
| **86** pages · 457 layout blocks | **26** Sitecore packages — 22 master + 4 targeted web exports | | |

Prepared
**August 2026**

Scope
**Sitecore export → Contentstack on Azure**

Diagrams
**18, drawn from the live export**

# Contents

The document has four parts. **A** follows one item from a Sitecore export to an entry on the Azure stack, phase by phase — nine short diagrams, each one mechanism. **B** is the content model, one domain per diagram. **C** is the set of rules that decide what becomes an entry, a block, a reference or nothing. **D** is what has been delivered and how it was proven.

How to read the diagrams: boxes are systems, steps or content types; an arrow is labelled with what moves along it; a dotted arrow is a rule or a check rather than data. Under every diagram, *How to read it* walks the boxes in order and *Why it matters* says what the picture decides.

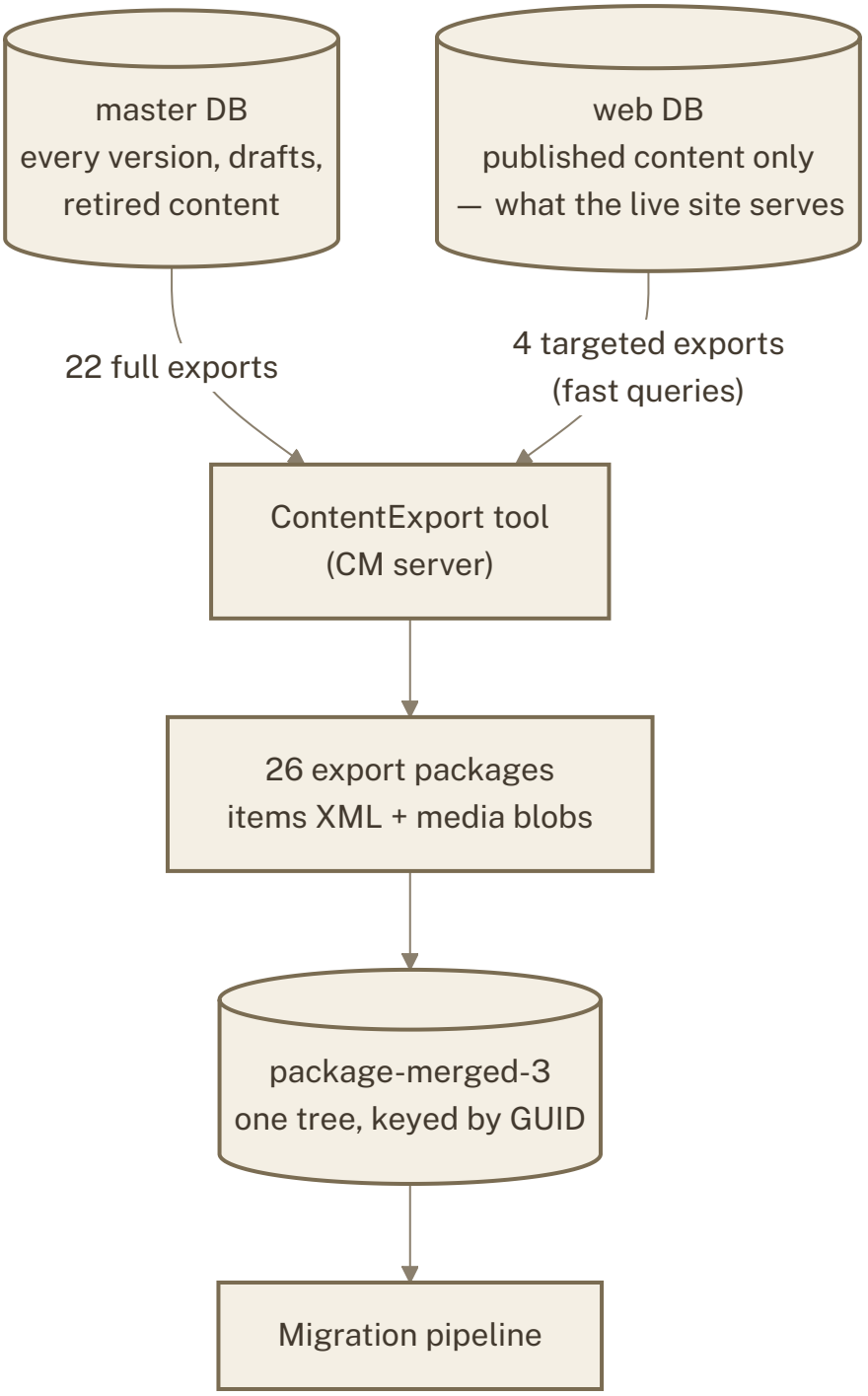
## Glossary — eight words used precisely

| | |
|---|---|
| `master / web` | Sitecore's two databases. **master** holds every authored version, drafts and retired content; **web** holds what was published. The live site serves from web. Exports default to master. |
| `package` | A Sitecore ContentExport zip: one XML per item, language and version, plus media blobs. 26 of them are merged into one working package, keyed by item GUID. |
| `mapper` | The parse of the package: Sitecore templates → Contentstack content-type rows with field types, reference targets, and the rules stamped on each row. |
| `FULL export` | The transform of every item in the package into entry JSON. Large; not what is imported. |
| `slice / carve` | The subset of FULL that the 98 live routes need, closed over references and children. This is the importable folder; the *carve* is the build step that produces it. |
| `delta` | Pushing only the difference between the export and a stack that already holds an earlier load — create what is new, patch what changed, never delete. |
| `mapping` | Export uid → stack uid for entries and assets (and export URL → stack URL). Written by the first import, extended by every delta; the record of what the stack holds. |
| `datasource` | Sitecore's pointer from a placement on a page to the item it renders. Becomes the `datasource` reference on a page's block. |

PART A · DATA FLOW

# A1 · Where content comes from

There is no database access to Sitecore. Content leaves it only through the ContentExport tool, as packages, from one of two databases.




**What it shows.** Two databases, one export path, one merged package.

HOW TO READ IT

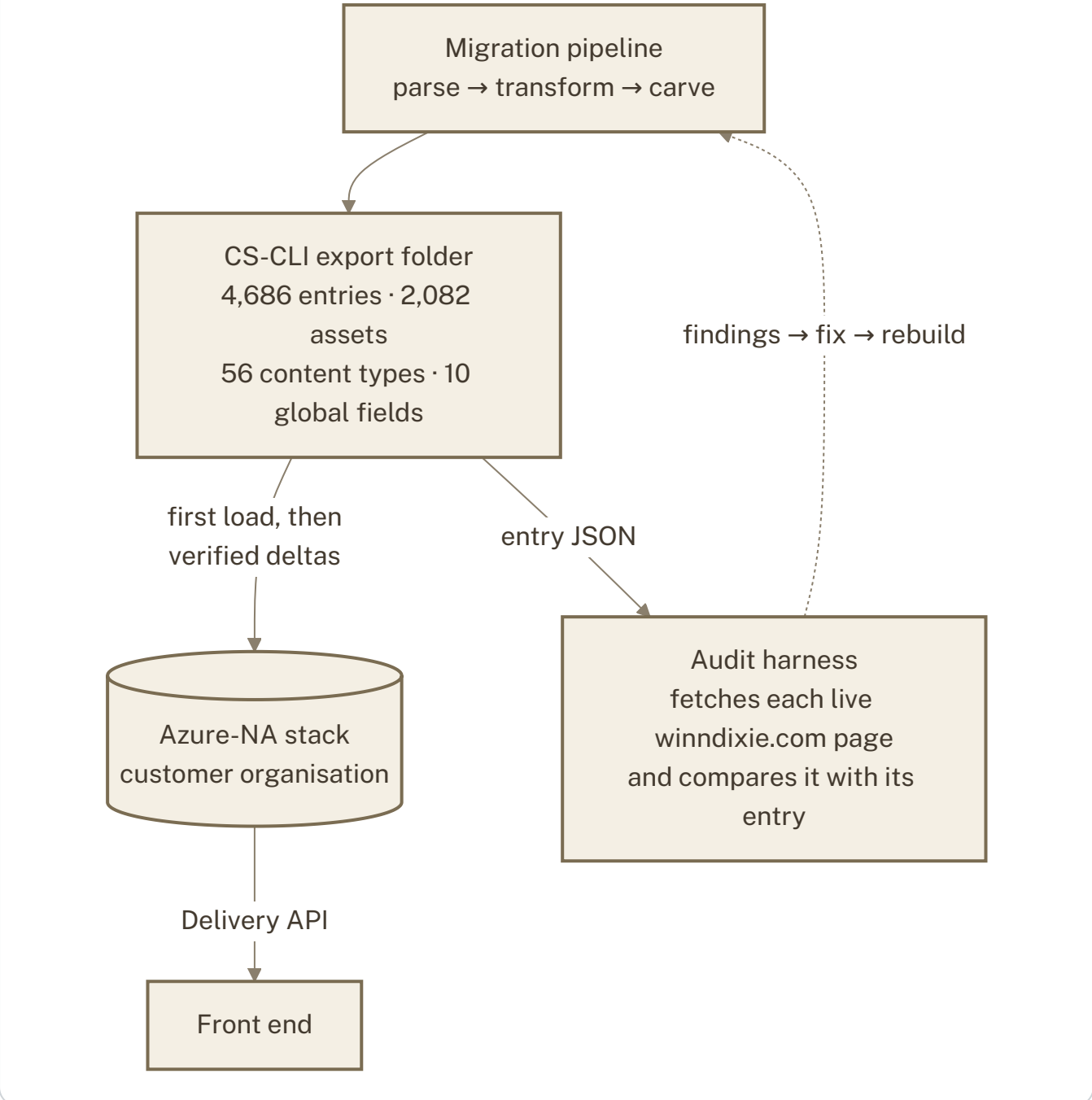
- **master** is where editors work: it has every version of every item, including drafts and things retired years ago.
- **web** is what publishing produces: the versions the live site actually serves. Some content exists only here (see A3).
- The **ContentExport tool** is the only exit. It produces a zip per run; a fast query can narrow a run to exactly the items that match.
- All 26 zips are merged into **one tree by item GUID**, so an item exported twice exists once.

WHY IT MATTERS Every "content is missing" question is first a question about which database it lives in. Recipe hero images, the contact form's labels and the Pet Types all lived only in web; they were found by comparing the export with the live site and recovered with targeted web exports.

PART A · DATA FLOW

# A2 · Where it goes and how it is checked

The pipeline produces a folder the Contentstack CLI can import. The customer stack receives it; the live site is the ground truth it is verified against; findings flow back into the pipeline rather than being patched on the stack.




**What it shows.** One export, two stacks with different roles, and the verification loop that closes on the live site.

HOW TO READ IT

- The **export folder** is the single artefact everything else is derived from and compared to.

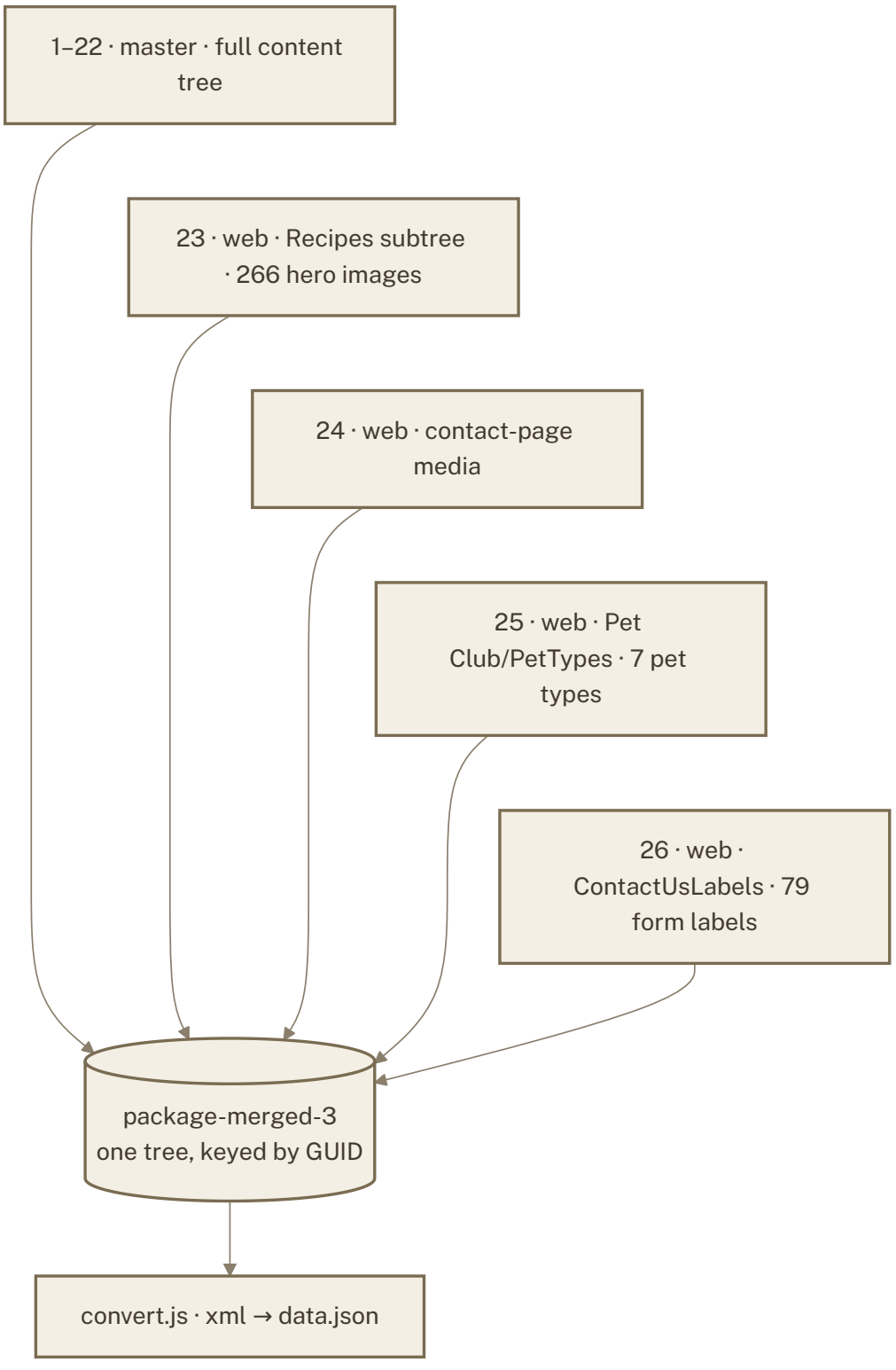
- **Azure-NA** is the customer organisation's stack — the delivery target. It received the first CLI import and, since then, only verified deltas (A9).
- A second stack, **NA**, is the sandbox for trials and next-scope experiments (not drawn), so the customer stack only ever sees verified content.
- The **audit harness** fetches each live page and compares it with the corresponding entry; a mismatch becomes a pipeline fix and a rebuild, never a hand edit.

WHY IT MATTERS The stack is reproducible from the export at any time; nothing on it depends on a manual step.

PART A · DATA FLOW

# A3 · Source packages and the web-database effort

The working package is a union of 22 master exports and four web exports, each of the latter recovering content the master database never had.



**What it shows.** Which parts of the tree came from web, and that the parser regenerates item JSON for them (a web export carries XML only).

HOW TO READ IT

- Packages **1–22** are the client's full master exports: 127 templates, ~20,000 items, 11,000 media items.
- **23** brought the recipe subtree from web — the hero image on 266 recipes existed only there.
- **24** brought the media folder behind the refreshed contact page.
- **25** brought the Pet Types master list; six master trees had zero items on that template.
- **26** brought the 79 contact-form labels the form controller reads; 37 of the form's 45 visible labels come from this folder.

WHY IT MATTERS The four web exports are the difference between a migration of what editors authored and a migration of what the site shows.

## The method behind each recovery

1. **Detect against live.**
   The audit harness compares winndixie.com with the export, page by page — a missing hero image, a form whose words never arrived, a template with zero items.

2. **Locate in web.**
   A Sitecore fast query on the web database (for example `fast:/sitecore/content//*[@@templatename='formlabels']`) confirms the items exist there and returns exactly them.

3. **Export and merge by GUID.**
   The targeted export is merged into the working package; the parser regenerates the item JSON; the build seeds the new items and links them where the site needs them.

4. **Prove.**
   Rebuild, diff against the previous build so that only the intended entries change, push as a delta, verify field by field on the stack.

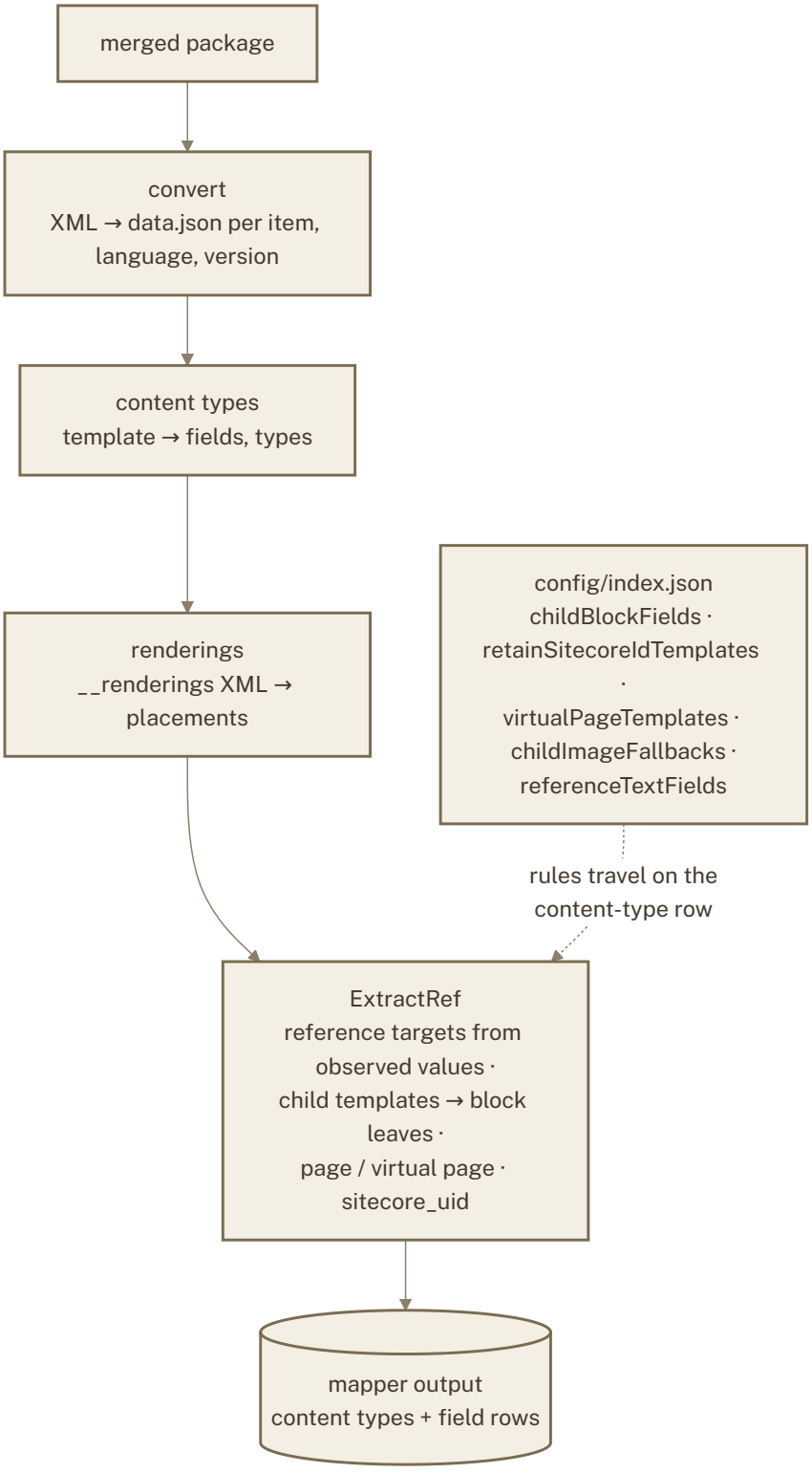
| 266 | 79 | 7 | 0 |
|---|---|---|---|
| recipe hero images recovered | contact-form labels → 43 of 45 form slots | Pet Types recovered | duplicates after four GUID merges |

**Note.** A fast-query export carries the matched items only — not their parent folder. Such items are seeded by the parent's GUID through the item graph, never by walking to a folder item that is not there.

PART A · DATA FLOW

# A4 · Phase 1 – Parse

The parser (upload-api) turns Sitecore templates into content-type definitions and works out, from the values items actually hold, what every reference field can point at. Config-driven rules are stamped onto the content-type row here.



**What it shows.** The four parse stages and the single point where configuration attaches to a content type.

HOW TO READ IT

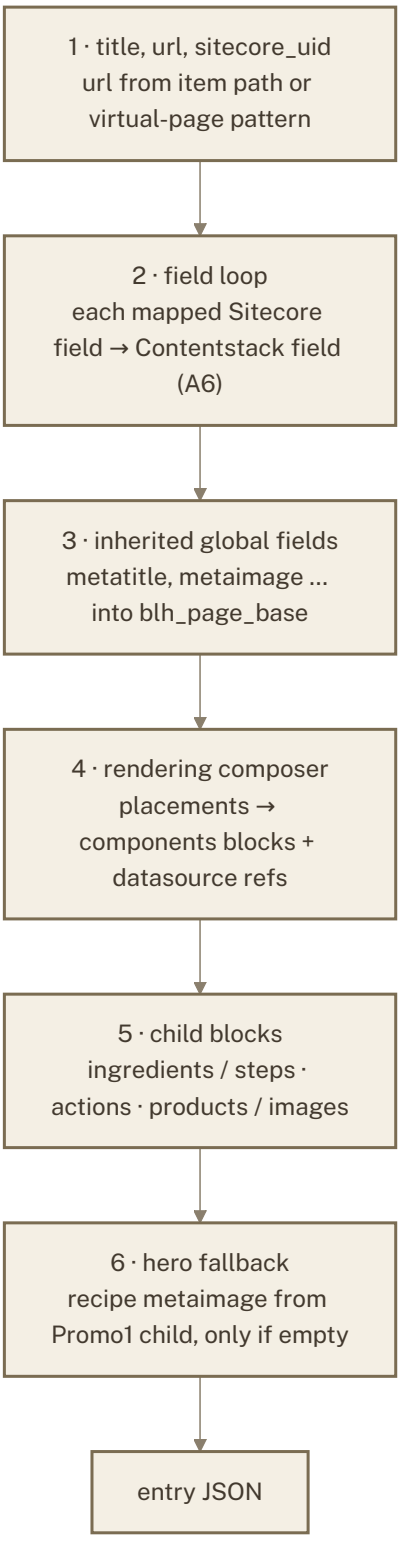
- **convert** reads every item's XML into JSON — one record per item, language and version.
- **content types** derives a Contentstack content type from each Sitecore template: field names, field types, groups.
- **renderings** reads each page's presentation XML into placements — which component, in which placeholder, pointing at which datasource.
- **ExtractRef** decides what every reference field may point at by reading the values items actually hold, marks which templates are pages, folds child templates into block definitions, and stamps the configured rules on the row.
- The **config** is read only here; the transformer reads the row, so a rule can never be applied in one layer and missed in the other.

WHY IT MATTERS Reference targets come from data, not from what an editor was allowed to pick. A field that points at nothing is left out rather than shipped as an invalid schema.

PART A · DATA FLOW

# A5 · Phase 2 — Transform: the six stages of one entry

The entry writer (api) assembles each entry in a fixed order. Later stages win over earlier ones by design.



**What it shows.** The order in which one entry is built; each stage may only fill what earlier stages left, except blocks, which are authoritative.

HOW TO READ IT

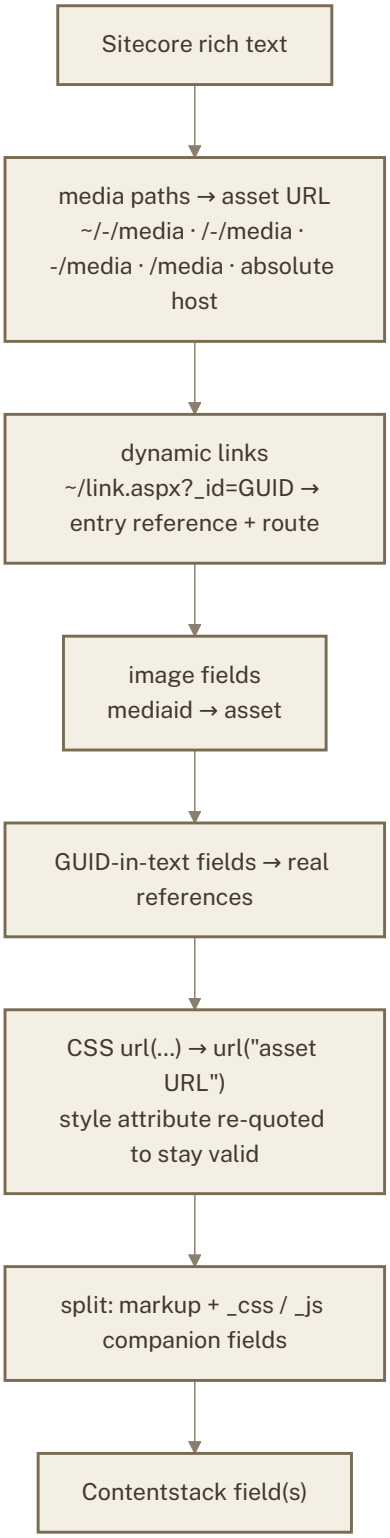
- **1** sets the identity: title, the route (from the item's path, or from a pattern for virtual pages such as recipes), and the Sitecore GUID where an external system keys on it.
- **2** copies every mapped field, applying the rich-text rewrites in A6.
- **3** fills SEO/meta fields a page inherits from its base template.
- **4** turns the page's placements into `components` blocks, each pointing at the entry it renders (B1).
- **5** folds child items into blocks — a recipe's ingredients, a promo's actions, a starter's products (B2).
- **6** supplies a recipe's hero image from its Promo1 child when the recipe itself has none.

WHY IT MATTERS A block always beats a same-named Sitecore field because the block is the denormalised child data the front end needs; the hero fallback never overwrites an authored image.

PART A · DATA FLOW

# A6 · Phase 2 — Transform: what happens to a rich-text field

Sitecore rich text is full of pointers into Sitecore — media paths, dynamic links, item GUIDs. Each is rewritten to its Contentstack counterpart so nothing on the stack depends on the old site.



**What it shows.** The six rewrites every rich-text value passes through, in order.

HOW TO READ IT

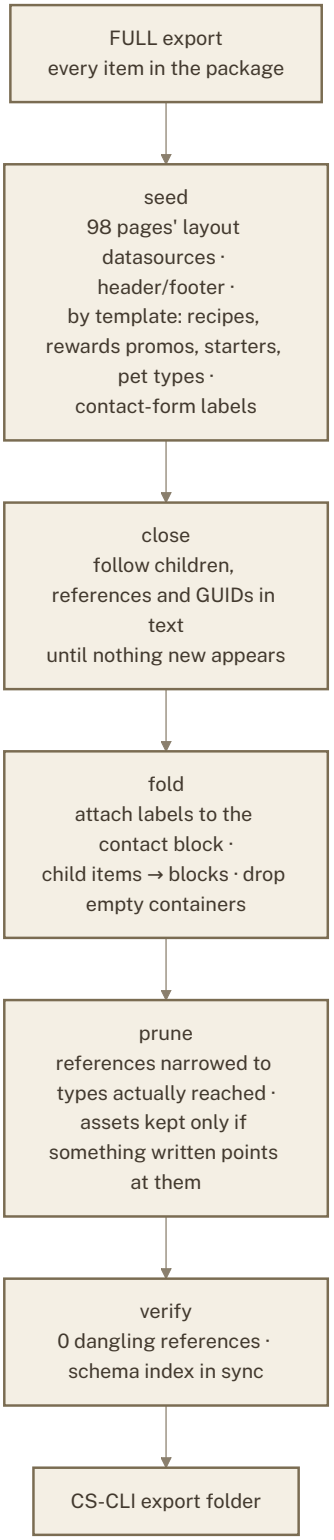
- **Media paths** in five Sitecore forms (and with the old site's host in front) become the asset's Contentstack URL.
- **Dynamic links** (`~/link.aspx?_id=…`) become a reference to the target entry plus its resolved route.
- **Image fields** resolve the media GUID to the uploaded asset.
- Fields Sitecore typed as text but filled with GUIDs become **real references**.
- **CSS** `url()` is rewritten to the asset URL, always double-quoted so the importer can remap it; a `style` attribute carrying one is re-quoted so the HTML stays valid.
- Inline **styles and scripts** are split into companion fields the editor can see.

WHY IT MATTERS Every image and link in migrated text resolves inside Contentstack; nothing points back at Sitecore hosts.

PART A · DATA FLOW

# A7 · Phase 3 — Carve: from everything to what the site needs

The FULL export is much bigger than what the live site uses. The carve keeps what the 98 live routes reach, plus the data the site fetches at runtime.



**What it shows.** Five steps from the full transform to the importable folder. The carve is rebuilt from scratch every run.

HOW TO READ IT

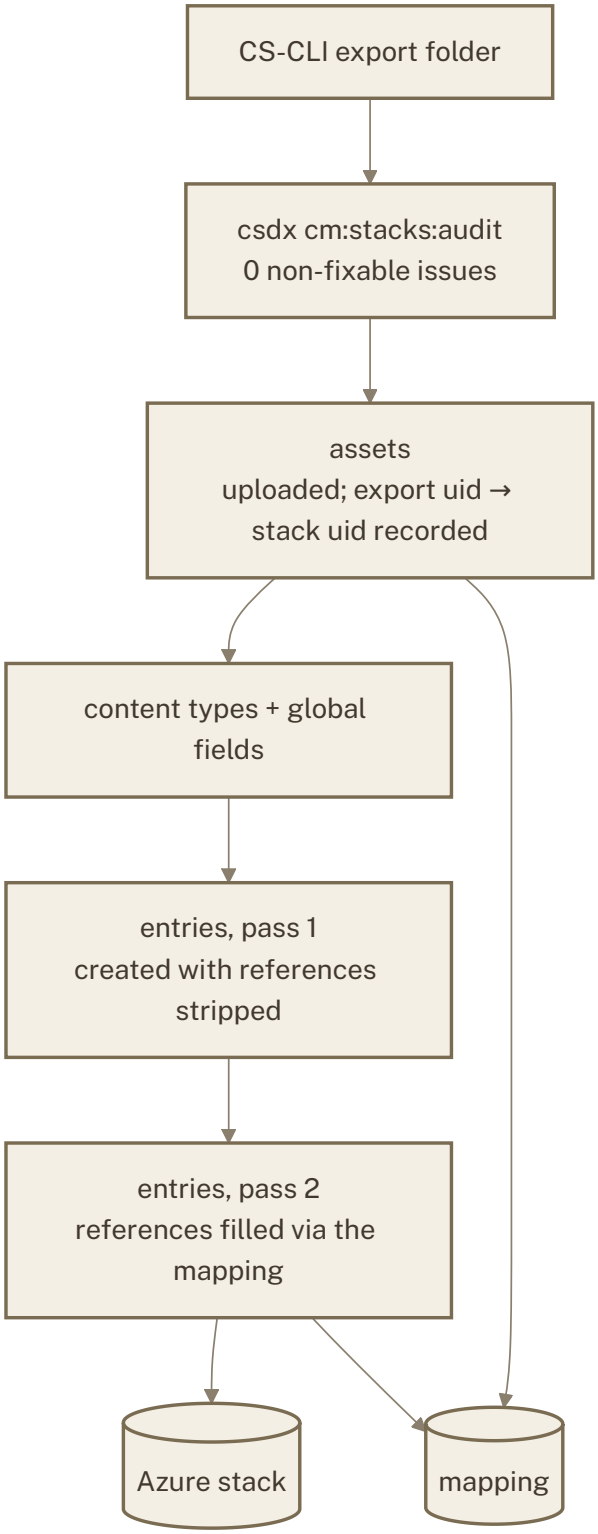
- **Seed** starts from what pages point at, not from the page tree — following children from Home would sweep the whole site in.
- **Close** follows real references, parent/child links and bare GUIDs in text (Sitecore stores some pointers that way).
- **Fold** applies the two denormalisations: path-read labels onto the contact block, child items into blocks (C1, C2).
- **Prune** narrows every reference field to the types the slice reaches and keeps only assets that something written uses.
- **Verify** fails the build on any dangling reference or schema mismatch.

WHY IT MATTERS The importable folder contains exactly what the site needs — no orphan entries, no unused assets, no reference to a type that does not ship.

PART A · DATA FLOW

# A8 · Phase 3 — First load

The first load went through the Contentstack CLI. It runs a stack audit first and writes the mapping that every later delta depends on.



**What it shows.** The CLI's order — assets, schemas, entries in two passes — and the mapping it leaves behind.

HOW TO READ IT

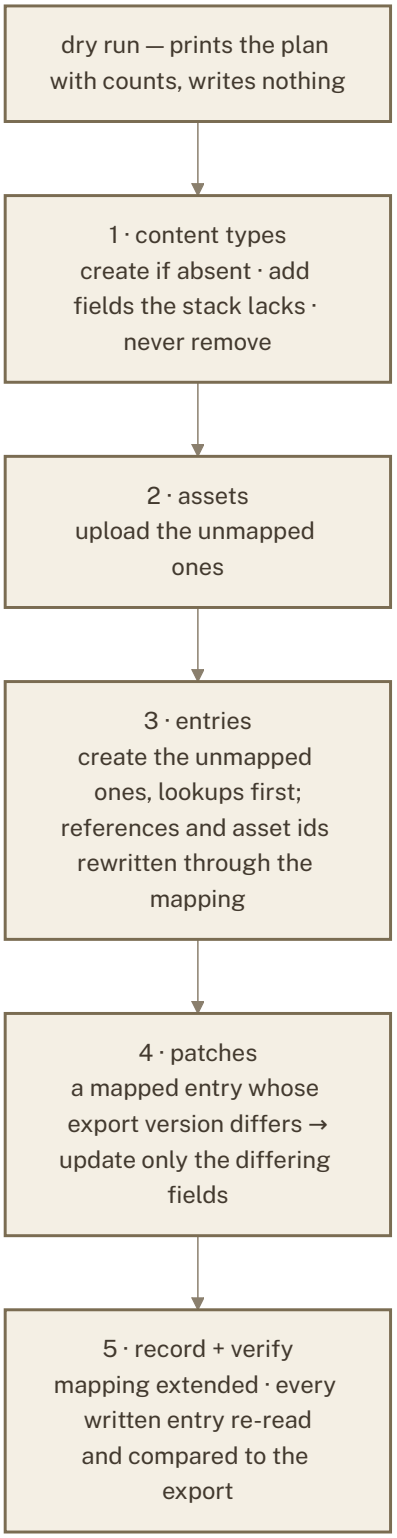
- The **audit** gate checks the folder for anything the stack would reject before a single write.
- **Assets** go first so entries can point at them; each upload records export uid → stack uid.
- **Schemas** next; then entries in **two passes**, because a reference cannot be set before its target exists.
- The **mapping** is the durable record of what the stack holds and what it is called there.

WHY IT MATTERS The mapping makes every later change a delta instead of a re-import.

PART A · DATA FLOW

# A9 · Delta pushes — keeping the stack in step with the export

After the first load nothing is re-imported. Each change moves as a delta: the export is the truth, the mapping says what the stack already has, only the difference moves, and nothing is ever deleted.



**What it shows.** The five ordered steps of a delta, always preceded by a dry run of the same plan.

HOW TO READ IT

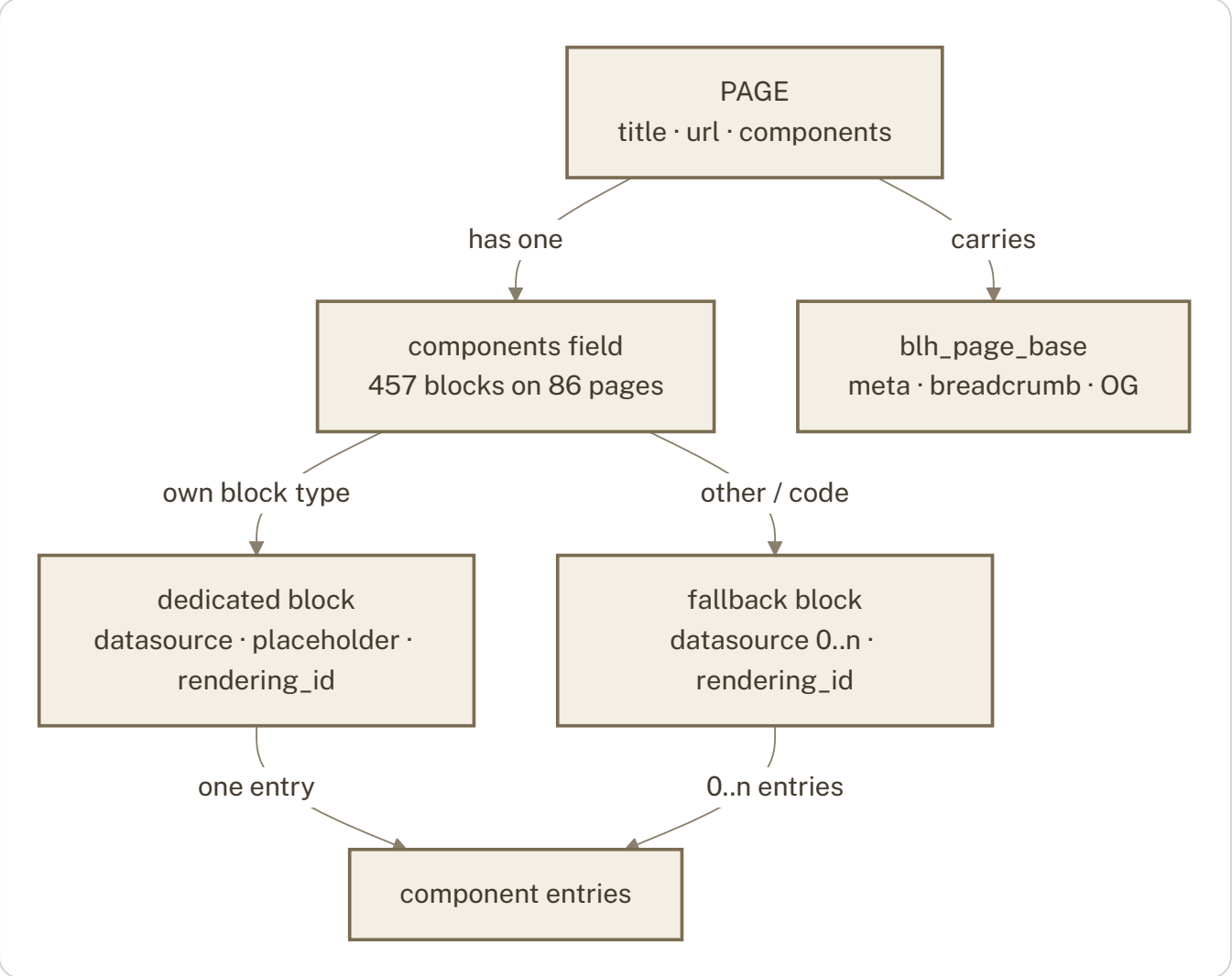
- A **dry run** prints this exact plan — how many content types, assets, entries, patches — and is reviewed before anything is applied.
- **Content types** are only ever extended; a field is never removed by a delta.
- **Entries** are created lookups-first so references resolve on the first pass; a reference to an entry created in the same run is filled in a second pass.
- **Patches** compare only the fields the export carries, ignoring the defaults Contentstack fills in, and can be limited to exactly the entries a change touched.
- **Verify** re-reads every written entry from the stack and compares it field by field.

WHY IT MATTERS Content authored directly in Contentstack is never overwritten: an export entry whose title already exists on the stack is skipped, and patches touch only what changed.

PART B · CONTENT MODEL

# B1 · How a page is composed

A Sitecore page's presentation becomes one modular-blocks field, `components`. Each placement becomes a block whose `datasource` reference points at the entry it renders.




**What it shows.** Page → blocks → the entries they render; two kinds of block, one mechanism.

HOW TO READ IT

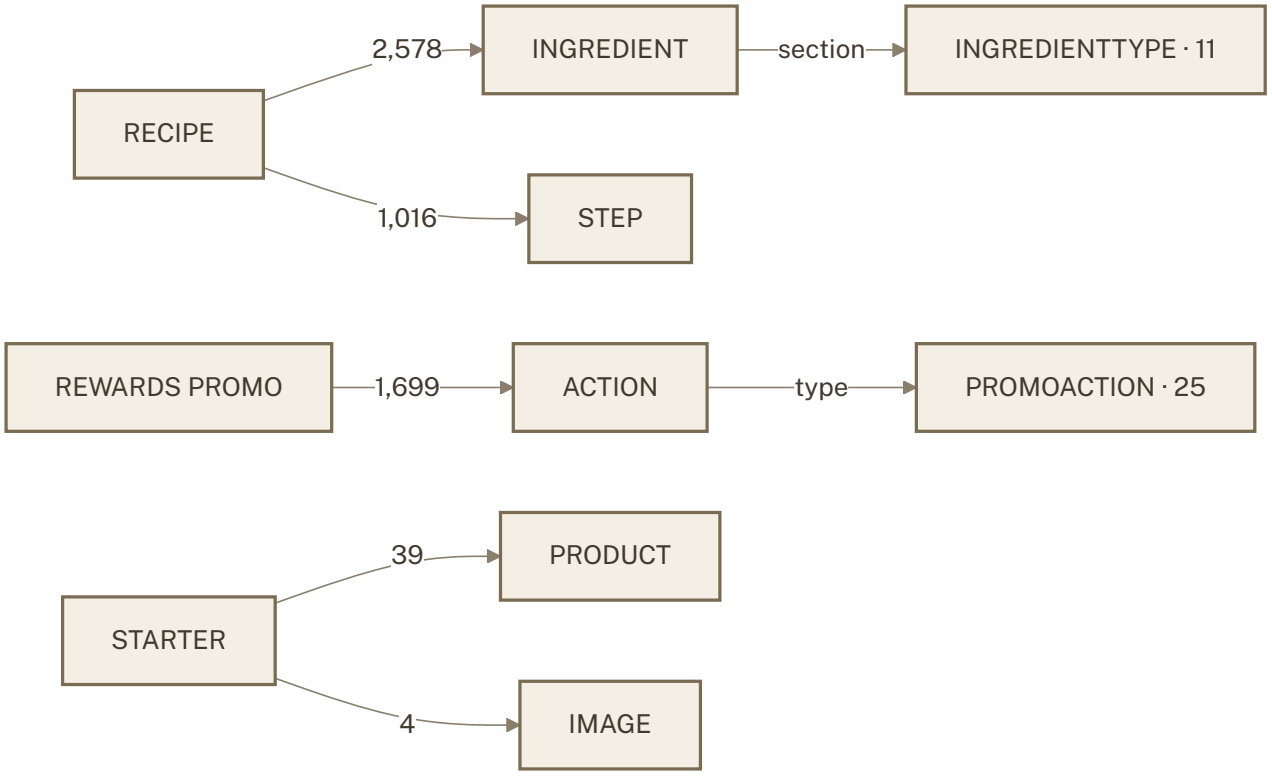
- A **page** (five page templates share this) has a `components` list in placement order.
- A **dedicated block** exists for each component type the pages use (20 on the main page type) and carries a single-type reference.
- The **fallback block** carries the leftovers — types Sitecore mixes into shared folders, and code-only renderings such as the store locator or the contact form — as a real reference where a datasource exists, and as position information (`rendering_id`, `placeholder`, `parameters`) where it does not.
- Every page also carries the **SEO/meta** global field.

WHY IT MATTERS The front end walks `components` in order and renders each block from its referenced entry — the same composition Sitecore had, without Sitecore's layout engine.

PART B · CONTENT MODEL

# B2 · Parent → child blocks

Where Sitecore stored "many small values under one item" as child items, Contentstack gets blocks on the parent. Three content types use it.




**What it shows.** The child items that became blocks, with their counts; each block keeps the child's Sitecore GUID.

HOW TO READ IT

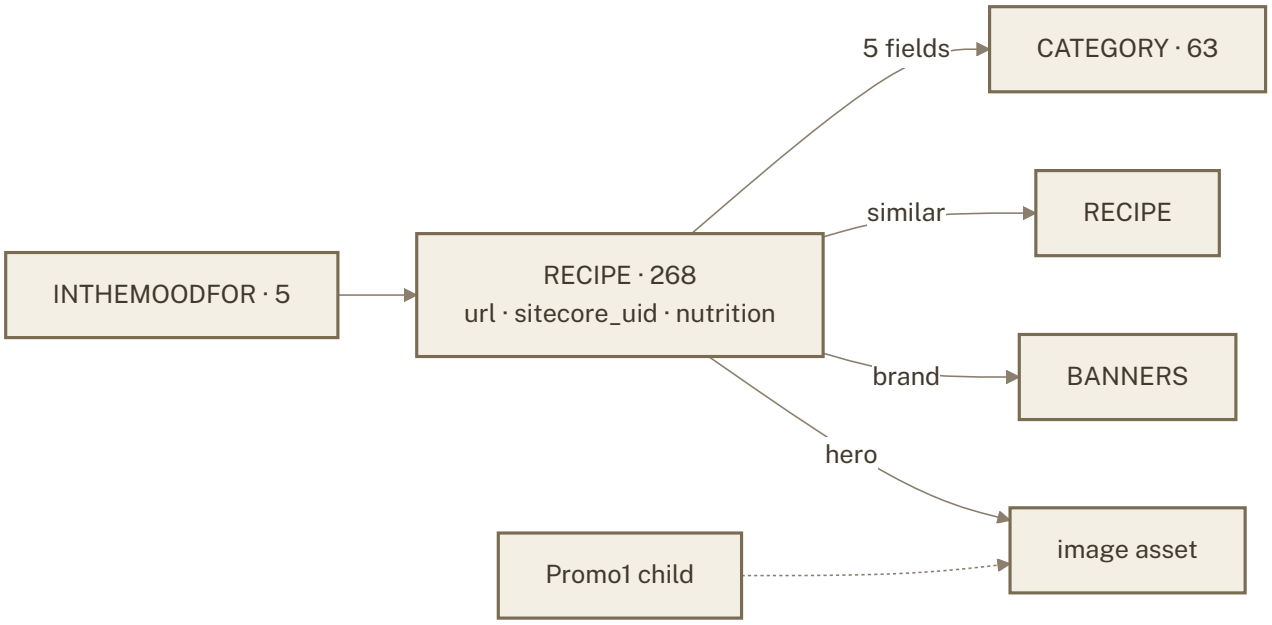
- A **recipe** carries its ingredients and steps in order; an ingredient may point at a **section heading** (Dough, Filling, Tempura Vegetables…).
- A **rewards promo** carries up to two **actions** — what the app opens when the card is tapped — each pointing at one of 25 action types.
- A **starter** carries its products (UPC, WD code) and images.
- Block fields: an ingredient carries name, quantity, unit and section; an action carries its type and a value (URL or GUID); a product carries UPC and WD code. Every block keeps `sitecore_uid`, the child item's GUID, because the external services that receive this data key on it.

WHY IT MATTERS 5,336 child items become blocks instead of standalone entries with meaningless titles; editors see a recipe with its ingredients, not 2,578 entries called `Ingredient1`.

PART B · CONTENT MODEL

# B3 · Recipes

Recipes have no layout of their own — the live site renders them through a shared details template at `/recipes/details/<slug>?recipeid=<guid>`. They are made pages anyway: URL from a pattern, GUID kept.




**What it shows.** 268 recipes, their five taxonomy fields, similar-recipe links, brand marker and the hero image's two sources (the recipe itself, or its Promo1 child).

HOW TO READ IT

- Five **category** fields classify a recipe; 63 category values ship, 11 ingredient-section headings alongside.
- **Similar recipes** were GUIDs in text fields in Sitecore; they are real references here.
- The **hero image** comes from the recipe's own fields, or from its `Promotions/Promo1` child when the recipe carries none — 266 of 268 recipes have one.
- **In the mood for** entries curate recipes on the landing page.

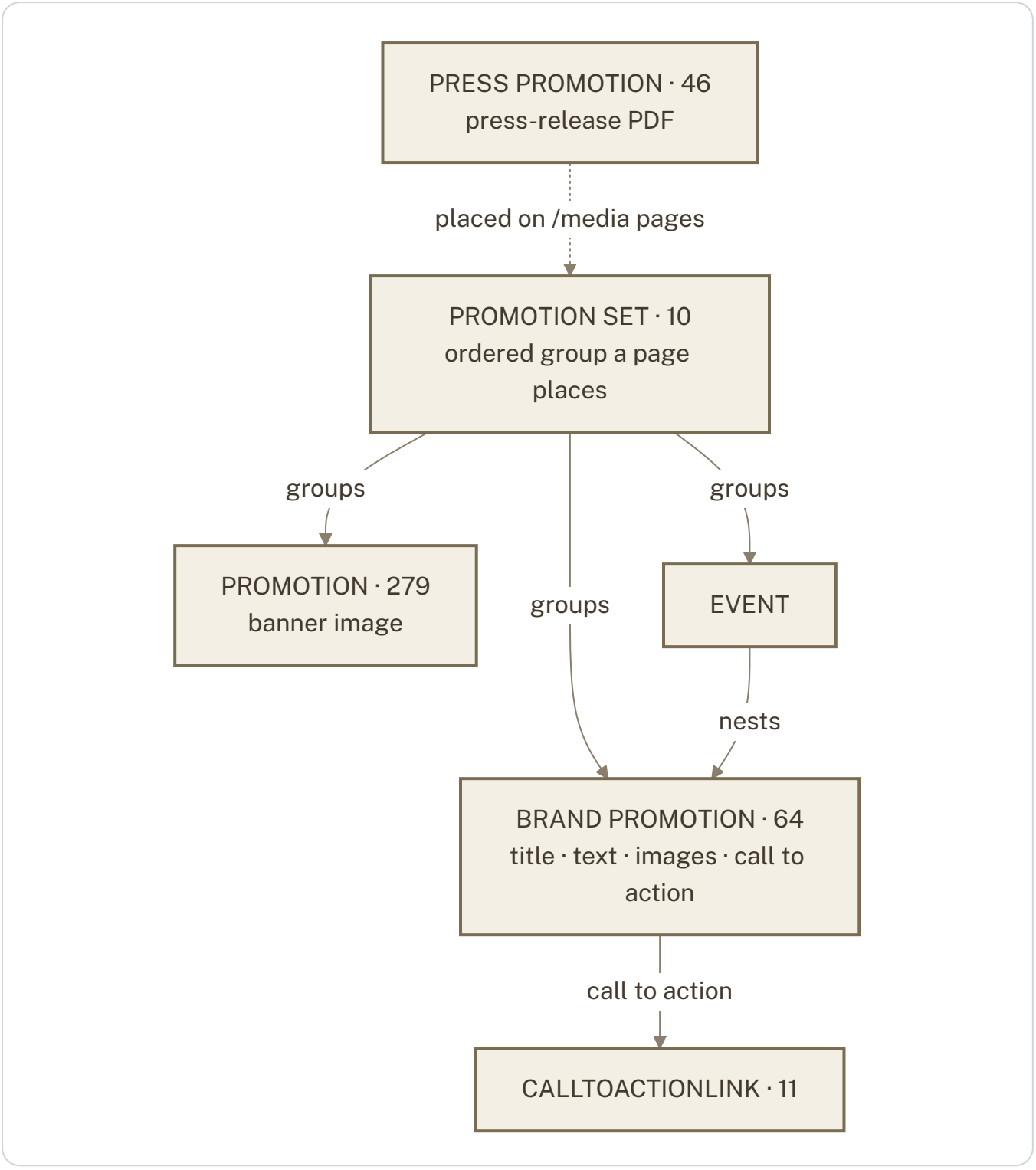
WHY IT MATTERS The recipe library is complete and addressable by the same GUID the live site uses, so existing links and the recipe automations keep working.

| CONTENT TYPE | ENTRIES | WHAT IT IS |
|---|---|---|
| `recipe` | 268 | Every live recipe; ingredients and steps are blocks (B2) |
| `category` | 63 | Taxonomy values across the five recipe fields |
| `ingredienttype` | 11 | Section headings inside a recipe's ingredient list |
| `inthemoodfor` | 5 | Curated callouts on the recipes landing page |

PART B · CONTENT MODEL

# B4 · Web promotions

Promotions are what pages place: single promos, brand cards, and the sets and events that group them.




**What it shows.** Sets group promos and events; events nest brand promos; every promo owns its images.

HOW TO READ IT

- A **promotion set** is an ordered group a page places; it may hold plain promotions, brand cards or events.
- A **brand promotion** is a card with title, text, image and a **call to action**.
- **Press promotions** are the 46 live press releases with their PDFs.

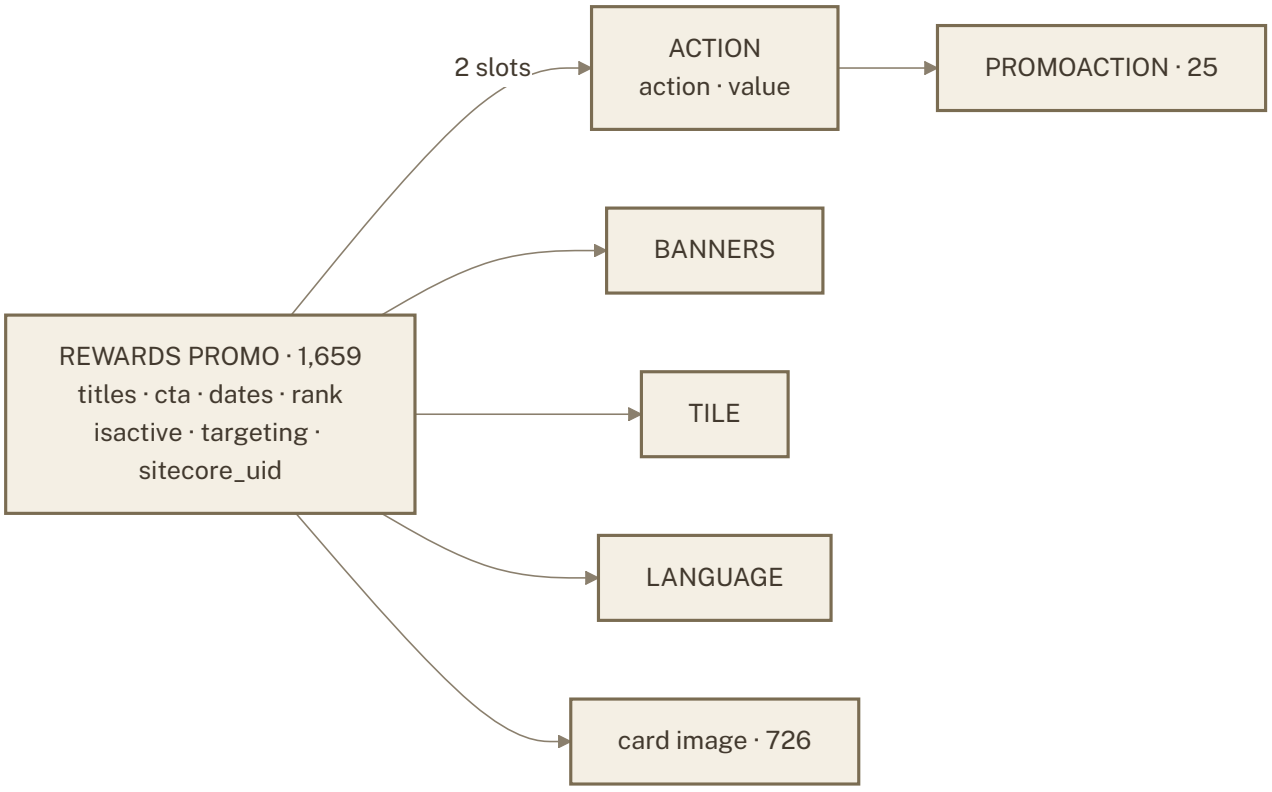
WHY IT MATTERS Editors reorder or swap what a page shows by editing the set, not the page.

| CONTENT TYPE | ENTRIES | WHAT IT IS |
|---|---|---|
| `promotion` | 279 | Web promotions placed on pages (includes recipe hero sources) |
| `brand_promotion` | 64 | Card-style promos with call to action |
| `promotion_set` / `promotionset` | 10 / 2 | Rich set (7 target types) and legacy set (promotion only) |
| `press_promotion` | 46 | Live press releases with PDF |
| `app_promotion` · `marqueebanner` · `digitalslide` | 10 · 6 · 37 | App banners, homepage marquee, weekly-ad slides |

PART B · CONTENT MODEL

# B5 · The mobile-app feed — rewards promos

`customerrewardspromo` is different in kind from web promotions: it is the weekly card slider of the mobile app, served by the loyalty API by brand, store, language and date window. No web page references it. It is carried in full — every card, every brand, every year — so the app team has the complete model and history.




**What it shows.** One card and everything the loyalty API reads from it — two action slots, a brand, a tile, a language and an image; the entry's shape matches the API payload field for field.

HOW TO READ IT

- A **card** has a title, subtitle, button text, image, a date window and a rank; flags and id lists target members or stores.
- Its **actions** say what the app opens on tap — one of 25 **action types** plus a value (a URL or an item GUID).
- **Brand**, **tile** and **language** are small lookups.
- `sitecore_uid` is the card's original GUID — the key the loyalty database already uses.

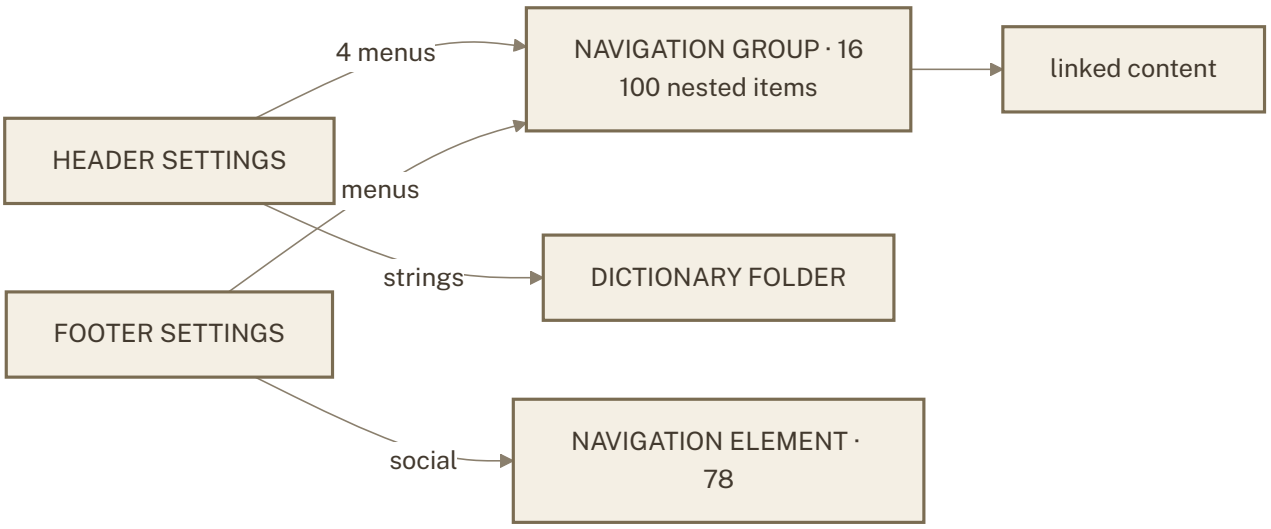
WHY IT MATTERS The Customer Rewards Promo automation can publish a card from Contentstack to the loyalty service with no field translation.

| CONTENT TYPE | ENTRIES | WHAT IT IS |
|---|---|---|
| `customerrewardspromo` | 1,659 | Cards 2021–2026 · WD 855 · HR 398 · FYM 405 · 1,699 action blocks · 726 images |
| `promoaction` | 17 | Action-type lookup (Rewards, DealsOfTheWeek, WeeklyAd, ActivateCoupon, ExternalURL, ListStarter …) |
| `banners` · `tile` · `language` | 6 · 2 · 2 | Lookups the cards point at |
| `starters` · `starterbanner` | 3 · 3 | List Starters for the app (products and images as blocks) and their brand lookup |
| `pettype` | 7 | Pet Club master list — Bird, Cat, Dog, Fish, Hamster, Other, Rabbit — with `isactive` and the original GUID |

PART B · CONTENT MODEL

# B6 · Navigation and site settings

Sitecore builds a menu from a tree of items; Contentstack gets one entry per root menu with the whole subtree as nested blocks. Header and footer are two singletons.




**What it shows.** Two settings singletons pointing at 16 menus; each menu carries its item tree as blocks and links onward to content.

HOW TO READ IT

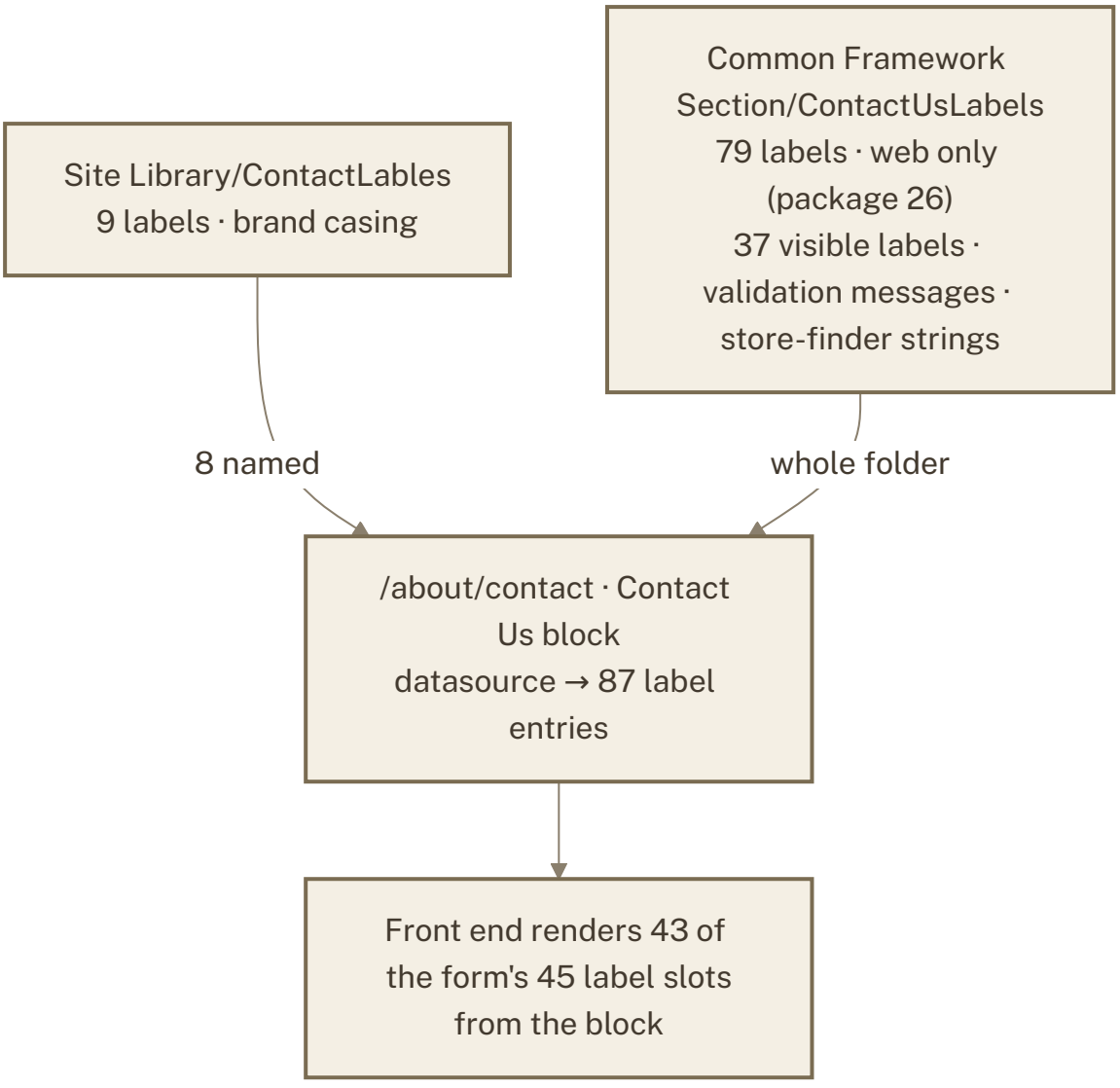
- **Header** and **footer settings** are one entry each; their pointer fields were plain text holding GUIDs in Sitecore and are real, typed references here.
- A **navigation group** is a whole menu: its items are nested blocks in Sitecore's order, each either a link or a piece of linked content.
- Each menu keeps its Sitecore GUID so the front end can resolve legacy pointers.

WHY IT MATTERS A menu is edited as one thing, and the header/footer schema says exactly what each field may point at.

PART B · CONTENT MODEL

# B7 · The contact form's words

The Contact Us form itself is code — fields, validation, reCAPTCHA live in the controller. Its *words* are Sitecore content the controller reads by folder path. Both label folders are attached to the page's Contact Us block so the front end resolves every label from the block.




**What it shows.** Content the code reads by path has no Sitecore pointer to mirror; the build attaches it to the block explicitly.

HOW TO READ IT

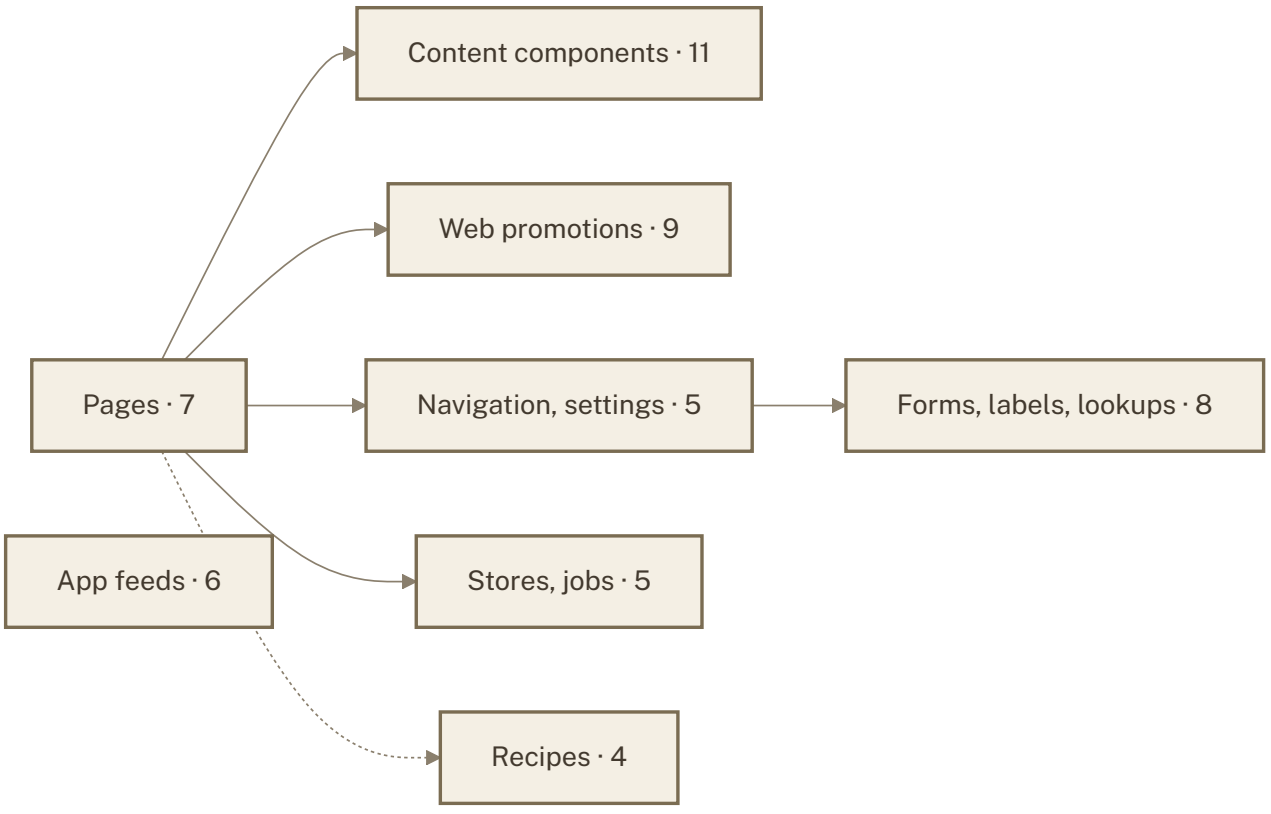
- The brand folder holds the 9 labels with the site's casing (*First name, Feedback category*); the shared folder holds the other 79 — field labels, validation messages, store-finder strings.
- All 87 are references on the contact page's Contact Us block, so the front end reads them from the block, not by matching text.
- Two of the 45 slots (*Rewards* as a feedback category, the thank-you message) exist nowhere in Sitecore and are authored in Contentstack.

WHY IT MATTERS Every word on the contact form is editable in Contentstack and reachable from the page that uses it.

PART B · CONTENT MODEL

# B8 · Domain map

The 56 content types with entries, grouped into eight domains, and how the domains reference each other.




**What it shows.** Solid arrows are block datasources from pages (or menu-item links); the dotted arrow marks recipes as virtual pages. Recipes and app feeds are self-contained; everything else hangs off pages.

HOW TO READ IT

- **Pages** reference every component domain through their blocks; nothing references pages.

- **Navigation** links to content and labels; **promotions** nest within themselves through sets and events.
- **Recipes** and **app feeds** are closed worlds: they reference only their own lookups, which is why they can be pushed and verified independently.
- Structural folders (`folder`, `branch_folder`, `template_folder`, 773 entries) mirror the Sitecore tree and are not drawn.

WHY IT MATTERS The delta creates lookups before the entries that point at them; this map is the order it follows.

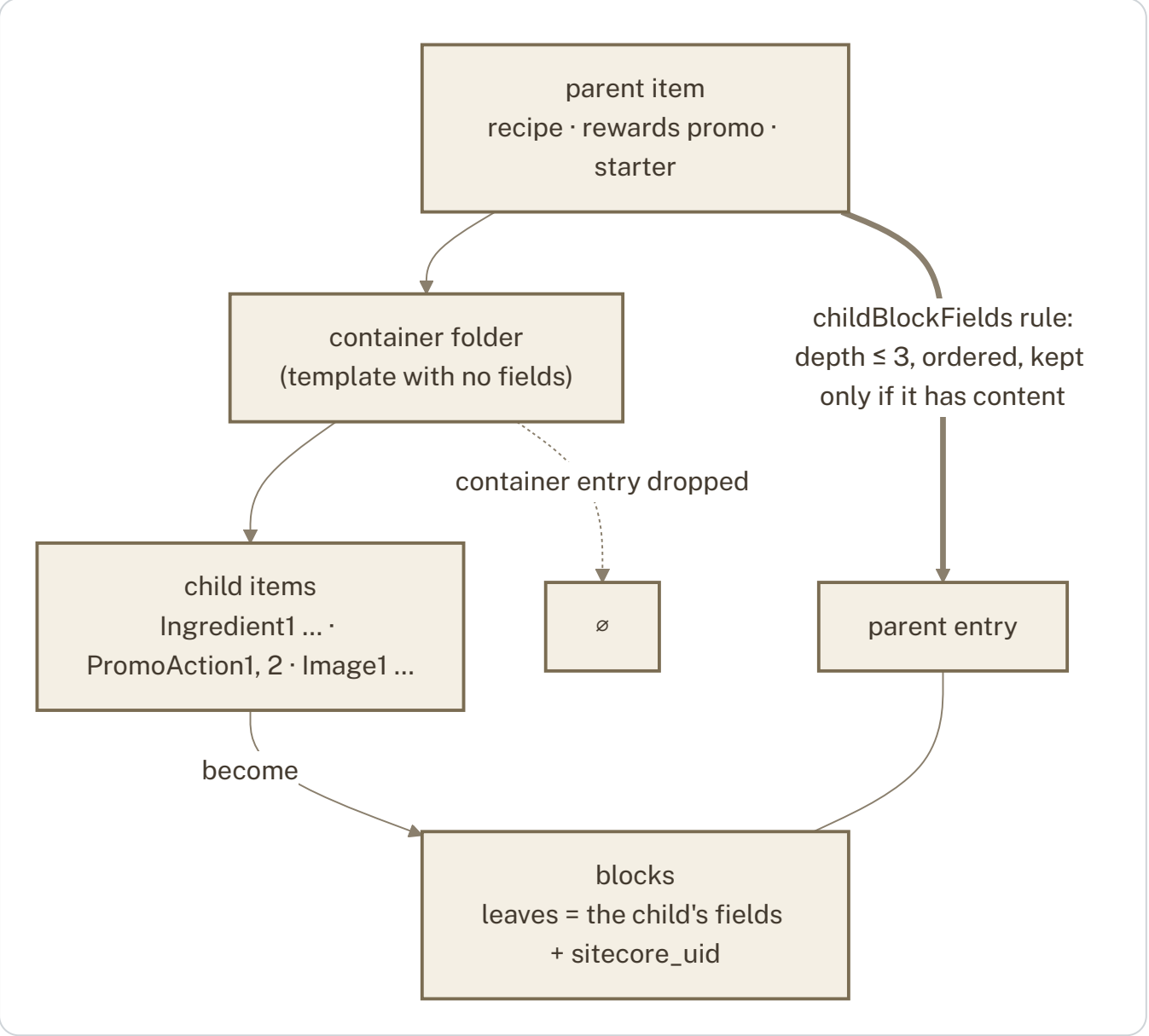
| DOMAIN | CONTENT TYPES |
|---|---|
| Pages (7) | generic_content_page, generic_left_navigation_content_page, template2/4/5/7_content_page |
| Content components (11) | page_title_and_text, page_title_text_and_image, page_title_text_link_and_image, page_content, htmltext, text, pageimage, calltoactionlink, instagram, digitalslide, marqueebanner |
| Web promotions (9) | promotion, promotion_set, promotionset, brand_promotion, app_promotion, press_promotion, signup_promotion, inthemoodfor, banners, tile |
| Navigation and settings (5) | navigation_group, navigation_element, site_header_settings, site_footer_settings, resetwelcombackmodaltemplate |
| Recipes (4) | recipe, category, ingredienttype, inthemoodfor |
| App feeds (6) | customerrewardspromo, promoaction, language, starters, starterbanner, pettype |
| Forms, labels, lookups (8) | formlabels, forminput, lookup_element, dictionary_entry, dictionary_folder, keyvalue, page_content_folder, folders |
| Stores and jobs (5) | storelocator, storelist, storedetails, jobsearch, featuredjobsset |

**Global fields (10):** `blh_page_base` (SEO/meta on every page), `page_content_base`, `call_to_action`, `promotion_base`, `promotion_set_base`, `isimageclickable`, `page_title_text_and_image_base`, `page_title_text_image_and_video_base`, `pageimage_base`, `aria_attributes`. **Assets (2,082)** attach through file fields on 29 content types and through rewritten rich-text references.

PART C · RULES

# C1 · Child items that are really fields

Sitecore models "many small values under one parent" as child items in a folder. Migrated literally they would be hundreds of standalone entries titled `Ingredient1` or `PromoAction2` that nothing can find. The rule folds them into blocks on the parent and ships no content type for the child template.




**What it shows.** One rule, three applications; the child's own fields become the block's fields, so a new field on the child template appears in the block without a code change.

HOW TO READ IT

- The **parent** is the entry that survives; the **container folder** between it and the children has no fields and is not carried.

- Each **child** becomes one block, in Sitecore's order (a sequence field where one exists, natural name order otherwise), and only if it carries content.
- A reference on a child (an action's type, an ingredient's section) is resolved before the child type is folded, so blocks carry real references.
- A picture on a child is collected as an asset just like one on a parent.

WHY IT MATTERS 5,336 child items are edited where they belong — inside their parent — and the external services receive them as the arrays they expect.

| PARENT | BLOCK FIELD | CHILD TEMPLATE | BLOCKS | ORDER | KEPT ONLY IF |
|---|---|---|---|---|---|
| `recipe` | `ingredients` | ingredients | 2,578 | sequence number | name or description |
| `recipe` | `steps` | steps | 1,016 | name | description |
| `customerrewardspromo` | `actions` | promoactions | 1,699 | name (PromoAction1, 2) | action or value |
| `starters` | `products` | starterproducts | 39 | name | UPC or WD code |
| `starters` | `images` | starterimages | 4 | name | image |

PART C · RULES

# C2 · Content the code reads by path

Some content has no rendering, no datasource and no reference pointing at it: the .NET controller reads a folder by path. Neither the graph walk nor the reference closure can reach it, so it is named explicitly.

| FOLDER | READ BY | SEEDED HOW | WHERE IT LANDS |
|---|---|---|---|
| `Site Library/ContactLables` | Contact Us controller | 8 named GUIDs | Contact Us block datasource on /about/contact |
| `Common Framework Section/ContactUsLabels` | Contact Us controller | folder GUID → children via the item graph | same block — 87 references in total |
| `RewardsPromo/Promos` | loyalty API (app slider) | by template | standalone entries, fetched by content type |
| `List Starters/Starters` | ListStarter API | by template | standalone |
| `Pet Club/PetTypes` | loyalty API (pet types) | by template | standalone |
| `Site Framework/Site Header · Footer Settings` | site layout | two singleton GUIDs | standalone; their menus closed over |

**One deliberate departure, documented.** Attaching the labels to the Contact Us block is the one place the export references something Sitecore did not. It was chosen because the front end resolves labels from block references; the alternative — unlinked entries matched by text — is fragile.

PART C · RULES

# C3 · The configuration rules in one table

| RULE | APPLIES TO | EFFECT |
|---|---|---|
| `childBlockFields` | recipe · customerrewardspromo · starters | Child items → blocks (C1); child content type not shipped |
| `retainSitecoreIdTemplates` | pettype · customerrewardspromo · starters | `sitecore_uid` stamped with the item GUID — external systems key on it (petTypeID, promo id, listStarterID) |
| `virtualPageTemplates` | recipe | A page without a layout: URL from a pattern, GUID retained |
| `childImageFallbacks` | recipe | Hero image from the Promo1 child when the recipe carries none |
| `referenceTextFields` | recipe · header/footer settings | GUID-in-text fields become typed references, split by target type |
| `renderingOptions` | all page templates | Placements → `components` blocks; fallback block for unmapped types |
| seeds by template | recipe · customerrewardspromo · starters · pettype | Runtime-fetched content included in full |
| `CONTACT_FORM` | /about/contact | Label folders attached to the Contact Us block |
| live press list | press_promotion | 46 live releases kept; 600 retired excluded with every reference to them |
| empty-container drop list | 7 container templates | Folder entries with no content and no reference are not shipped |

PART C · RULES

# C4 · What is in, what is out, and why

| ITEM | DECISION | WHY |
|---|---|---|
| Live routes (98) with their components | IN | Scope rule: what is live on winndixie.com |
| Recipes 268 + categories + ingredient types | IN | Fetched at runtime by the recipe selector; the whole library is wanted |
| Rewards promos — all 1,659 with 726 images | IN | Complete model and history for the app team |
| List Starters, Pet Types | IN | Feeds for the Automate integrations; recovered from the web database |
| Contact-form labels, 87 | IN | The form's authored words; 43 of 45 slots resolve from the block |
| `sitecore_uid` on pet types, promos, starters | ADDED | External databases key on the Sitecore GUID |
| Recipe ingredient section headings | RECOVERED | 9 recipes regained their grouping (e.g. Tempura Vegetables / Miso-Mustard Sauce) |
| Placeholder copy in 7 entries | KEPT | Authored in the current Sitecore versions; carried faithfully so editors can replace it (the live site does not render these sections today) |
| es-ES language versions | NEXT | Ready in the package (3 starters, 78 contact labels); carried when the Spanish locale is enabled |
| Instagram feed cache and API tokens | OUT | An API cache, not content — the site fetches reels live from Instagram; credentials belong in the front end's secret store |
| Template-level chrome (NotificationBar, CookieBar ...) | FRONT END | Site-wide chrome the front end owns; pages carry their own renderings |
| 600 retired press releases | OUT | Not live; the 46 live ones are carried with their PDFs |

PART D · STATE

# D1 · Delivered to the Azure stack

Everything in the export is on the customer stack, reconciled content type by content type. Where the stack holds more than the export, the extra entries were authored directly in Contentstack by the team.

| CONTENT TYPE | EXPORT | AZURE | NOTE |
|---|---|---|---|
| `customerrewardspromo` | 1,659 | 1,659 | with actions blocks and `sitecore_uid` |
| `recipe` | 268 | 268 | ingredient section headings restored on 9 |
| `generic_content_page` | 86 | 87 | +1 page authored directly in Contentstack |
| `formlabels` | 102 | 102 | 87 linked on /about/contact |
| `brand_promotion` | 64 | 70 | +6 department promos authored directly in Contentstack |
| `pettype` · `starters` · `starterbanner` | 7 · 3 · 3 | 7 · 3 · 3 | — |
| assets | 2,082 | 2,089 | every asset URL verified HTTP 200; +13 uploaded directly |
| content types | 69 | 69 | schemas identical, field by field |

PART D · STATE

# D2 · Verification standard

The same proof was applied to the first load and to each of the seven deltas since.

1. **Dry run.**
   The plan is printed with counts — content types to add, assets to upload, entries to create, entries to patch — and nothing is written.

2. **Apply.**
   Only the planned writes happen; nothing is ever deleted.

3. **Re-read.**
   Every created or patched entry is fetched back from the stack and compared field by field to the export.

4. **Idempotency.**
   A second dry run must plan zero writes — the stack and the export agree.

5. **Live check.**
   Counts per content type against the stack API; asset URLs fetched; sampled pages compared with winndixie.com.

**Result today.** 4,686 entries and 2,082 assets in the export; every touched content type on Azure matches it exactly; 0 dangling references; 0 broken asset URLs.